messaging routes. With this framework, the consumer can create the contracts and share them in multiple different ways (check https://mng.bz/EOqO for the examples). Using the framework stub runner, the provider implementation generates the stubs that can be used with the contracts to verify that both work together. Some of the great features from this framework include automatically generating boilerplate test code and enabling stubs to be used by other projects.

Spring Cloud also provides a way to write contracts with YAML allowing applications in other programming languages to use the Spring Cloud JVM implementation. You don't need to understand Java to use this framework, but you do need to be able to work with YAML, which can be tricky at times too with indentation problems.

BENEFITS OF SPRING CLOUD CONTRACT

- Native support within JVM projects
- Multilanguage support with YAML files

DOWNSIDE OF SPRING CLOUD CONTRACT

- Contracts packaged in the provider repository

A.1.8 Postman

Postman providers can produce an OpenAPI blueprint, and consumers can build their requests and share the collection with the provider (https://mng.bz/Dpqw). This is a great way to use a tool you may be familiar with and gain some of the benefits of contract testing. As discussed previously, one of the main benefits of consumer-driven contract testing (CDCT) is enabling communication between distributed teams. In addition, the opportunity for consumers to supply the provider team with test scenarios removes any assumptions and unknowns around how the API is being used. Provider-driven contract testing (PDCT) is also a way to use tools such as Postman, allowing you to use your existing collections to verify against the OpenAPI specification.